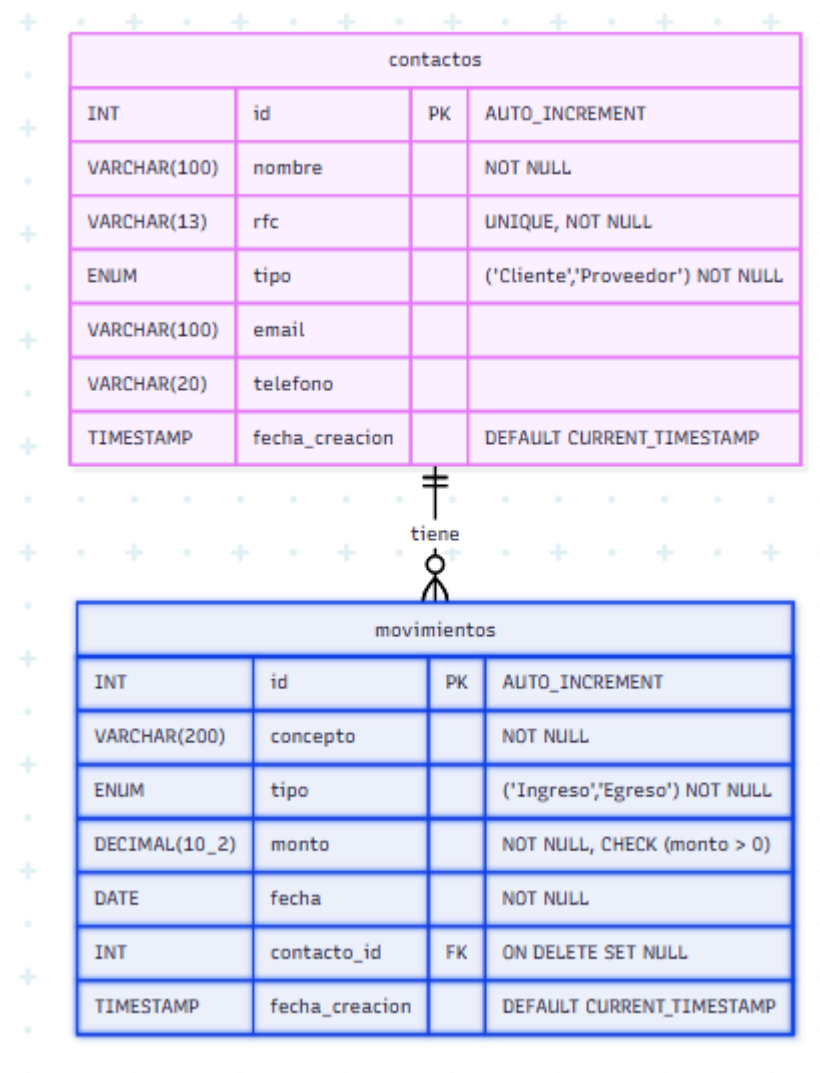


****Nombre y apellido del estudiante:**** Edwards Pérez
****Cédula de identidad:**** 32533151
****Carrera:**** Informática - 5to Semestre
****Unidad Curricular:**** Interfaces Web con el Usuario
****Docente:**** Ing. José Daniel Cadenas L.
****Fecha de entrega:**** Jueves 01/10/2026
****Semana:**** 9

1. DIAGRAMA ENTIDAD-RELACIÓN



****Descripción breve del modelo:****

El modelo cuenta con dos entidades principales: `contactos` (que almacena tanto clientes como proveedores) y `movimientos` (que registra ingresos y egresos de caja o banco). La relación es de 1 a muchos (1:N) entre `contactos` y `movimientos`, ya que un contacto puede tener múltiples movimientos asociados, pero cada movimiento pertenece, a lo sumo, a un contacto. Se utiliza la restricción `ON DELETE SET NULL` para asegurar que si se

elimina un contacto, los movimientos no se pierdan, quedando huérfanos pero contabilizados.

2. DOCUMENTACIÓN DE LAS 4 PETICIONES

2.1 Crear cliente (POST /api/contactos)

- **Descripción del endpoint:** Crea un nuevo registro en la tabla `contactos`.

- **Request:**

- **Método:** POST

- **URL:** `http://localhost:3000/api/contactos`

- **Headers:** `Content-Type: application/json`

- **Body:**

```
```json
{
 "nombre": "Distribuidora Los Andes C.A.",
 "rfc": "DLA2026100101",
 "tipo": "Cliente",
 "email": "ventas@andes.com",
 "telefono": "0412-1234567"
}
```

- **Response:** Status Code `201 Created`

```
```json
{
  "exito": true,
  "mensaje": "Contacto creado exitosamente",
  "datos": {
    "id": 3,
    "nombre": "Distribuidora Los Andes C.A.",
    "rfc": "DLA2026100101",
    "tipo": "Cliente"
  }
}
```

HTTP <http://localhost:3000/api/contactos> Save Share

POST <http://localhost:3000/api/contactos> Send

One gateway for every AI and API call. Skip the one-off integrations — route LLMs, MCP servers, and APIs through Fabric instead. [Learn more about Fabric](#)

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "nombre": "Distribuidora Los Andes C.A.",
3   "rfc": "DLA2026100101",
4   "tipo": "Cliente",
5   "email": "ventas@andes.com",
6   "telefono": "0412-1234567"
7 }
8
```

Body 201 Created • 62 ms • 426 B

{ JSON Preview Visualize

```
1 {
2   "exito": true,
3   "mensaje": "Contacto creado exitosamente",
4   "datos": {
5     "id": 3,
6     "nombre": "Distribuidora Los Andes C.A.",
7     "rfc": "DLA2026100101",
8     "tipo": "Cliente"
9   }
10 }
```

2.2 Registrar movimiento (POST /api/movimientos)

- **Descripción del endpoint:** Inserta un nuevo flujo de caja o banco en la tabla `movimientos`.

- **Request:**

- **Método:** POST

- **URL:** `http://localhost:3000/api/movimientos`

- **Headers:** `Content-Type: application/json`

- **Body:**

```
``json
{
  "concepto": "Venta de servicios de consultoría",
  "tipo": "Ingreso",
```

```
"monto": 2500.00,  
"fecha": "2026-10-01",  
"contacto_id": 3  
}  
...
```

- **Response:** Status Code `201 Created`

```
``json  
{  
  "exito": true,  
  "mensaje": "Movimiento registrado",  
  "datos": {  
    "id": 3,  
    "concepto": "Venta de servicios de consultoría",  
    "tipo": "Ingreso",  
    "monto": 2500.00,  
    "fecha": "2026-10-01"  
  }  
}  
...
```

HTTP <http://localhost:3000/api/movimientos> Save Share

POST <http://localhost:3000/api/movimientos> Send

One gateway for every AI and API call. Skip the one-off integrations — route LLMs, MCP servers, and APIs through Fabric instead. [Learn more about Fabric](#)

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "concepto": "Venta de servicios de consultoría",
3   "tipo": "Ingreso",
4   "monto": 2500.00,
5   "fecha": "2026-10-01",
6   "contacto_id": 3
7 }
8
```

Body 201 Created • 7 ms • 439 B • ...

{ } JSON Preview Visualize

```
1 {
2   "exito": true,
3   "mensaje": "Movimiento registrado",
4   "datos": {
5     "id": 3,
6     "concepto": "Venta de servicios de consultoría",
7     "tipo": "Ingreso",
8     "monto": 2500,
9     "fecha": "2026-10-01"
10  }
11 }
```

No anomalies detected

2.3 Consultar datos (GET /api/movimientos)

- **Descripción del endpoint:** Devuelve el listado de todos los movimientos registrados junto con el nombre del contacto asociado (JOIN).
- **Request:**
 - **Método:** GET
 - **URL:** `http://localhost:3000/api/movimientos`
 - **Headers:** Ninguno
- **Response:** Status Code `200 OK`

```
"exito": true,
"datos": [
```

```
{
  "id": 3,
  "concepto": "Venta de servicios de consultoría",
  "tipo": "Ingreso",
  "monto": "2500.00",
  "fecha": "2026-10-01T04:00:00.000Z",
  "contacto_id": 3,
  "fecha_creacion": "2026-10-01T13:56:39.000Z",
  "contacto_nombre": "Distribuidora Los Andes C.A."
},
{
  "id": 2,
  "concepto": "Compra de insumos",
  "tipo": "Egreso",
  "monto": "450.50",
  "fecha": "2026-09-16T04:00:00.000Z",
  "contacto_id": 2,
  "fecha_creacion": "2026-10-01T13:52:15.000Z",
  "contacto_nombre": "Proveedor Global"
},
{
  "id": 1,
  "concepto": "Ajuste inicial Cédula: 32533151",
  "tipo": "Ingreso",
  "monto": "1500.00",
  "fecha": "2026-09-15T04:00:00.000Z",
  "contacto_id": 1,
  "fecha_creacion": "2026-10-01T13:52:15.000Z",
  "contacto_nombre": "Edwards Pérez"
}
]
}
```

http://localhost:3000/api/movimientos

GET http://localhost:3000/api/movimientos

200 OK • 9 ms • 956 B •

Body

```
1 {
2   "exito": true,
3   "datos": [
4     {
5       "id": 3,
6       "concepto": "Venta de servicios de consultoría",
7       "tipo": "Ingreso",
8       "monto": "2500.00",
9       "fecha": "2026-10-01T04:00:00.000Z",
10      "contacto_id": 3,
11      "fecha_creacion": "2026-10-01T13:56:39.000Z",
12      "contacto_nombre": "Distribuidora Los Andes C.A."
13    },
14    {
15      "id": 2,
16      "concepto": "Compra de insumos",
17      "tipo": "Egreso",
18      "monto": "450.50",
19      "fecha": "2026-09-16T04:00:00.000Z",
20      "contacto_id": 2,
21      "fecha_creacion": "2026-10-01T13:52:15.000Z",
22      "contacto_nombre": "Proveedor Global"
23    },
24    {
25      "id": 1,
26      "concepto": "Ajuste inicial Cédula: 32533151",
27      "tipo": "Ingreso",
28      "monto": "1500.00",
29      "fecha": "2026-09-15T04:00:00.000Z",
30      "contacto_id": 1,
31      "fecha_creacion": "2026-10-01T13:52:15.000Z",
32      "contacto_nombre": "Edwards Pérez"
33    }
34  ]
35 }
```

2.4 Generar reporte (GET /api/resumen)

- **Descripción del endpoint:** Devuelve los totales de ingresos, egresos, saldo neto y número total de movimientos.

```
- **Request:**  
- **Método:** GET  
- **URL:** `http://localhost:3000/api/resumen`  
- **Headers:** Ninguno  
- **Response:** Status Code `200 OK`
```

```
{  
  "exito": true,  
  "datos": {  
    "totalIngresos": 4000,  
    "totalEgresos": 450.5,  
    "saldo": 3549.5,  
    "totalMovimientos": 3  
  }  
}
```

HTTP <http://localhost:3000/api/resumen> Save Share

GET <http://localhost:3000/api/resumen> Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

raw JSON Schema Beautify

1

Body 200 OK • 7 ms • 372 B

{ } JSON Preview Visualize

```

1  {
2    "exito": true,
3    "datos": {
4      "totalIngresos": 4000,
5      "totalEgresos": 450.5,
6      "saldo": 3549.5,
7      "totalMovimientos": 3
8    }
9  }

```

3. TABLA DE CÓDIGOS DE ESTADO HTTP

Código	Significado	Uso en el ERP
200	OK	Consultas exitosas (ej. GET /api/contactos) y PUT exitoso.
201	Created	Recursos creados exitosamente (ej. POST /api/contactos).
400	Bad Request	Validaciones fallidas (campos vacíos, montos negativos, RFC duplicado).

| 404 | Not Found | Recursos no encontrados (ej. ID de contacto que no existe). |
| 500 | Server Error | Error interno, fallo de conexión a MySQL, bug en el código. |

4. VALIDACIÓN DE PRECISIÓN CONTABLE

Tabla de cálculos manuales (basado en los pasos del ejercicio 7.1):

Concepto	Cálculo manual
Total Ingresos	$1500.00 + 2500.00 = \$4,000.00$
Total Egresos	$450.50 + 200.00 = \$650.50$
Saldo	$4000.00 - 650.50 = \$3,349.50$
Total movimientos	4

HTTP client interface showing a POST request to `http://localhost:3000/api/movimientos`. The request body is a JSON object:

```
1 { "concepto": "Venta inicial", "tipo": "Ingreso", "monto": 1500.00,
2   "fecha": "2026-10-01", "contacto_id": 1 }
```

The response status is **201 Created** (7 ms, 418 B). The response body is a JSON object:

```
1 {
2   "exito": true,
3   "mensaje": "Movimiento registrado",
4   "datos": {
5     "id": 4,
6     "concepto": "Venta inicial",
7     "tipo": "Ingreso",
8     "monto": 1500,
9     "fecha": "2026-10-01"
10  }
11 }
```

HTTP

http://localhost:3000/api/movimientos

Save

Share

POST

http://localhost:3000/api/movimientos

Send

Docs

Params

Authorization

Headers 8

Body

Scripts

Settings

Cookies

raw

JSON

Schema

Beautify

```
1 { "concepto": "Compra insumos", "tipo": "Egreso", "monto": 450.50,
2   "fecha": "2026-10-01", "contacto_id": 2 }
```

Body

201 Created

6 ms

419 B

⌕

⋮

{}

JSON

Preview

Visualize

⌕

≡

🔍

📄

🔗

```
1 {
2   "exito": true,
3   "mensaje": "Movimiento registrado",
4   "datos": {
5     "id": 5,
6     "concepto": "Compra insumos",
7     "tipo": "Egreso",
8     "monto": 450.5,
9     "fecha": "2026-10-01"
10  }
11 }
```

HTTP <http://localhost:3000/api/movimientos> Save Share

POST <http://localhost:3000/api/movimientos> Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

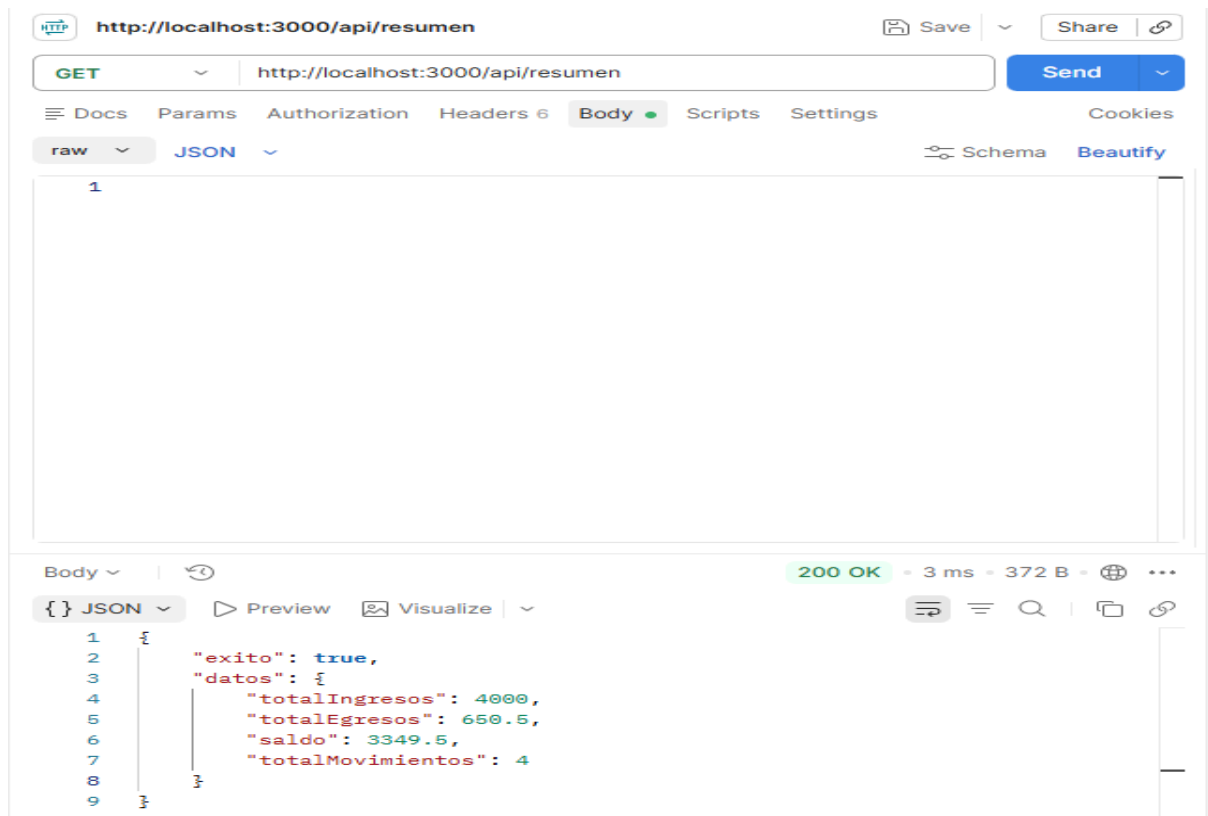
raw JSON Schema Beautify

```
1 { "concepto": "Venta consultoria", "tipo": "Ingreso", "monto": 2500.00,
2   "fecha": "2026-10-01", "contacto_id": 1 }
```

Body 201 Created = 25 ms = 423 B

{ } JSON Preview Visualize

```
1 {
2   "exito": true,
3   "mensaje": "Movimiento registrado",
4   "datos": {
5     "id": 6,
6     "concepto": "Venta consultoria",
7     "tipo": "Ingreso",
8     "monto": 2500,
9     "fecha": "2026-10-01"
10  }
11 }
```



****Respuesta del endpoint /api/resumen:****

```
{
  "exito": true,
  "datos": {
    "totalIngresos": 4000,
    "totalEgresos": 650.5,
    "saldo": 3349.5,
    "totalMovimientos": 4
  }
}
```

****Conclusión de la comparación:****

Al contrastar los cálculos aritméticos realizados de forma manual con la respuesta en formato JSON generada por el endpoint /api/resumen, he podido verificar una coincidencia absoluta en los montos totales (Ingresos: \$4,000.00 / Egresos: \$650.50 / Saldo: \$3,349.50). Esto confirma que el motor de base de datos MySQL está procesando correctamente las funciones de agregación (SUM, COUNT) y las condicionales (CASE) incluidas en la consulta SQL. Además, demuestra que la lógica de negocio aplicada en el backend (parseo a flotante y resta de totales) es robusta y precisa para un sistema financiero.

5. PREGUNTAS REFLEXIVAS

****Pregunta 1 — Sobre el DER****

Explica la relación entre las entidades contactos y movimientos. ¿Por qué elegiste cardinalidad 1:N y no N:M? ¿Qué pasaría si un movimiento pudiera tener múltiples contactos? ¿Cómo cambiaría el modelo de datos?

****Respuesta:**** Elegí cardinalidad 1:N porque un movimiento (una factura o un recibo) normalmente está asociado a un único cliente o proveedor específico. Si la relación fuera N:M, implicaría que un solo movimiento podría dividirse entre múltiples entidades, lo cual requeriría una tabla intermedia (`contacto\_movimiento`) para registrar qué porción del monto le pertenece a quién, lo que haría la consulta de saldos individuales más compleja.

****Pregunta 2 — Sobre persistencia****

Compara el comportamiento de tu servidor en la Semana 7-8 (datos en memoria RAM) con el de la Semana 9 (datos en MySQL). ¿Qué ventajas reales observaste al migrar? Menciona al menos 3 ventajas concretas.

****Respuesta:**** En las semanas anteriores los datos se perdían cada vez que nodemon reiniciaba el servidor, ahora la persistencia es permanente, lo cual es obligatorio en producción. Tres ventajas clave: 1) Los datos no se pierden al reiniciar. 2) Integridad garantizada mediante restricciones y llaves foráneas a nivel de base de datos. 3) Consultas complejas más óptimas como agrupaciones (SUM), delegando ese procesamiento al motor de base de datos en lugar de usar ciclos en JavaScript.

****Pregunta 3 — Sobre códigos HTTP****

Si un estudiante intenta crear un movimiento con monto negativo, ¿qué código HTTP debería recibir y por qué? Explica la diferencia entre un error 400 (Bad Request) y un error 500 (Server Error) en el contexto de este ERP.

****Respuesta:**** Debería recibir un código 400 (Bad Request) porque es un error del cliente al mandar datos que violan una regla de negocio del sistema. La diferencia radica en la responsabilidad: el error 400 indica que la petición estaba mal estructurada o contenía datos inválidos (como un monto menor a 0), mientras que un 500 indica un fallo interno en nuestro código o pérdida de conexión con MySQL.

****Pregunta 4 — Sobre validaciones****

¿Por qué es importante validar los datos en el backend (Express) y no solo en el frontend (Vue)? Da un ejemplo concreto de qué pasaría si un usuario malintencionado envía datos inválidos directamente a la API saltándose el frontend.

****Respuesta:**** Es vital porque el frontend puede ser manipulado o interceptado (e.g. usando Postman). Si un usuario malintencionado mandara directamente una petición a la API con un monto de -50000 como ingreso (sin validación en backend), alteraría nuestro saldo de la empresa inyectando datos corruptos que el motor contable asimilaría como reales.

****Pregunta 5 — Sobre precisión contable****

En el Ejercicio 7 comparaste cálculos manuales con la respuesta del endpoint `/api/resumen`. ¿Por qué es crítica esta validación en un sistema financiero? ¿Qué consecuencias tendría un error de redondeo o cálculo en un sistema real de tesorería?

****Respuesta:**** En un sistema financiero, los errores de redondeo se propagan. Un céntimo mal calculado en millones de transacciones se traduce en faltantes (o sobrantes) de miles de dólares en las cuentas bancarias reales. La precisión contable y las validaciones de tipo (e.g. asegurar parseos correctos y uso de DECIMAL) aseguran que el sistema informático refleje fielmente la realidad del dinero que posee la organización.

6. PRUEBA DE FUEGO - DEFENSA EN VIVO

Reto #1: Cambio de Puerto

Enunciado del docente: "Apaga tu servidor. Cambia el puerto de escucha de 3000 a 3005 en tu código y haz que funcione."

Pasos que realicé:

Abrí el archivo principal del servidor `server.js`.

Busqué la constante `PORT` (al inicio del archivo) que estaba definida en 3000.

Cambié su valor a 3005 para que la aplicación Express escuche en ese nuevo puerto al ejecutarse.

Código modificado:

```
javascript
const app = express();
const PORT = 3005; // <-- Línea modificada
```

Resultado: Funcionó a la primera. El servidor se reinició y la consola mostró que ahora escucha peticiones en `http://localhost:3005`. Captura de pantalla:

HTTP <http://localhost:3005/api/movimientos> Save Share

GET <http://localhost:3005/api/movimientos> Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Body 200 OK • 59 ms • 1.11 KB • ...

{ } JSON Preview Visualize

```
2  { "exito": true,
3    "datos": [
4      {
5        "id": 1,
6        "concepto": "Ajuste inicial Cédula: 32533151",
7        "tipo": "Ingreso",
8        "monto": "1500.00",
9        "fecha": "2026-10-01T04:00:00.000Z",
10       "contacto_id": 1,
11       "fecha_creacion": "2026-10-01T14:10:59.000Z",
12       "contacto_nombre": "Edwards Pérez"
13     },
14     {
15       "id": 2,
16       "concepto": "Compra de insumos",
17       "tipo": "Egreso",
18       "monto": "450.50",
19       "fecha": "2026-10-01T04:00:00.000Z",
20       "contacto_id": 2,
21       "fecha_creacion": "2026-10-01T14:10:59.000Z",
22       "contacto_nombre": "Proveedor Global"
23     },
24     {
25       "id": 3,
26       "concepto": "Venta consultoría",
27       "tipo": "Ingreso",
28       "monto": "2500.00",
29       "fecha": "2026-10-01T04:00:00.000Z",
30       "contacto_id": 1,
31       "fecha_creacion": "2026-10-01T14:11:49.000Z",
32       "contacto_nombre": "Edwards Pérez"
33     },
34     {
35       "id": 4,
36       "concepto": "Servicios internet",
37       "tipo": "Egreso",
38       "monto": "200.00",
39       "fecha": "2026-10-01T04:00:00.000Z",
40       "contacto_id": 2,
```

¿Qué aprendí?: Aprendí que el puerto donde escucha una API en Node.js es simplemente una variable de configuración de Express, y es muy fácil de reasignar.

Reto #2: Consulta con Filtro (Reto 5 de la lista original)

Enunciado del docente: "Modifica el endpoint GET /api/movimientos para que acepte un parámetro ?tipo=Ingreso y solo devuelva los movimientos de ese tipo."

Pasos que realicé:

En server.js, ubiqué el endpoint `app.get('/api/movimientos')`.

Extraje el parámetro tipo desde `req.query` (`const { tipo } = req.query`).

Modifiqué la consulta SQL estática para convertirla en dinámica: si viene el parámetro tipo, le concateno una cláusula `WHERE m.tipo = ?` y le paso el valor en el arreglo de parámetros para evitar inyecciones SQL.

Código modificado:

javascript

```
app.get('/api/movimientos', async (req, res) => {
  try {
    const { tipo } = req.query; // Capturamos el parámetro de la URL

    let sqlQuery = `
      SELECT m.*, c.nombre AS contacto_nombre
      FROM movimientos m
      LEFT JOIN contactos c ON m.contacto_id = c.id
    `;

    const params = [];

    if (tipo) {
      sqlQuery += ` WHERE m.tipo = ?`; // Filtro dinámico
      params.push(tipo);
    }

    sqlQuery += ` ORDER BY m.fecha DESC`;
    const [rows] = await db.query(sqlQuery, params);
    res.status(200).json({ exito: true, datos: rows });
  } catch (error) {
    // ...
  }
});
```

Resultado: Funcionó sin errores. Al agregar `?tipo=Ingreso` al final de la URL en Postman, el servidor retornó un array filtrado que contiene exclusivamente los ingresos de dinero.

Captura de pantalla:

```
{
  "exit": true,
  "datos": [
    {
      "id": 1,
      "concepto": "Ajuste inicial Cédula: 32533151",
      "tipo": "Ingreso",
      "monto": "1500.00",
      "fecha": "2026-10-01T04:00:00.000Z",
      "contacto_id": 1,
      "fecha_creacion": "2026-10-01T14:10:59.000Z",
      "contacto_nombre": "Edwards Pérez"
    },
    {
      "id": 3,
      "concepto": "Venta consultoría",
      "tipo": "Ingreso",
      "monto": "2500.00",
      "fecha": "2026-10-01T04:00:00.000Z",
      "contacto_id": 1,
      "fecha_creacion": "2026-10-01T14:11:49.000Z",
      "contacto_nombre": "Edwards Pérez"
    }
  ]
}
```

¿Qué aprendí?: Aprendí a extraer variables de la URL (Query Params) con `req.query` y a usarlas para armar sentencias SQL dinámicas de forma segura concatenando strings y usando los placeholders `?` en el `db.query()`.

7. CONCLUSIONES

A través de este taller, he comprendido de manera práctica la vital importancia que tiene la persistencia de datos en el desarrollo backend. Mientras que en semanas anteriores utilizábamos arreglos en memoria que provocaban la pérdida total de la información ante cualquier reinicio del servidor, la integración de MySQL con Node.js y Express mediante el paquete `mysql2` me permitió construir una aplicación mucho más robusta y realista. Aprendí que delegar el procesamiento de datos al motor SQL (utilizando funciones como `SUM` o `JOIN`) no solo optimiza el rendimiento, sino que también garantiza la integridad referencial de nuestra información financiera a través de las llaves foráneas. Finalmente, enfrentarme a la 'Prueba de Fuego' me ayudó a consolidar mi capacidad de leer, comprender y modificar

código asíncrono bajo presión, entendiendo cómo capturar parámetros tanto en el cuerpo de la petición (body) como en la URL (query params), habilidad esencial para cualquier desarrollador backend.